Parameter-Efficient Fine-Tuning: A Systematic Comparison of Efficient Adaptation Methods

Abhishek Singh Bisht
abhishekbisht322@gmail.com

Shashank Viswakarma
shakuvish07@gmail.com

***Abstract*—This paper reviews five parameter-efficient fine-tuning methods: Full fine-tuning BERT model, BitFit, LoRA, IA^3, and Prefix tuning, applied to GLUE tasks under conditions of constrained training data. We used 30%, 50%, 70%, and 100% data fractions. BitFit surprisingly performed well under low data conditions. LoRA almost achieved full fine-tuning performance while it used less than 1% of the parameters. Prefix tuning was somewhat erratic at first but settled down when it was trained for enough epochs. IA^3 displayed steady improvement, but for the most part, it did not stand out as the best option.**

The analysis goes on to indicate that with increased amounts of training data, the performance gap between full fine-tuning and these other efficient techniques tends to close. In other words, at practically no point do parameter-efficient approaches, with all their pros and cons, take their less-than-ideal position to full fine-tuning. Parameter-efficient techniques exhibit different stability profiles, highlighting that the choice one makes depends specifically on traded-off needs and constraints.

***Index Terms*—Parameter-efficient fine-tuning, Model adaptation, Training dynamics, Fine-tuning stability**

I. Introduction

Fine-tuning large transformer models [1] for different tasks has become standard practice in NLP. Parameter-efficient fine-tuning (PEFT) methods [2] have emerged over the past few years as a potential solution. Instead of adapting the entire model, these techniques identify small subsets of parameters or structures that can somehow capture the needed part for a specific task [3]. Hence, evident advantages are lower memory requirements, faster training, and less complicated handling. However, the raw empirical reality behind this is much more complicated than it seems. Most published studies emphasize final validation accuracy and overlook critical aspects of training dynamics, data efficiency, and the stability of the convergence process [4] [5] the very aspects that are really important when these methods want to be applied in practice.

In this paper, we considered five different approaches: BERT full fine-tuning, BitFit [6], LoRA [7], IA^3 [8], and prefix tuning [9]. This was applied to three GLUE tasks: CoLA [10], MRPC [11], and STS-B [12] where we intentionally varied the amount of training data from 30% to 100% and tracked performance over epochs. The experimental setup allows the observations of not only the final performance but also the progression of training, especially under data-starved conditions.

We found that BitFit, despite being really simple, sometimes matches full fine-tuning when working with moderate amounts of data. LoRA performs well while staying parameter-efficient. Prefix tuning does have some instability issues early in training, but it's not a fundamental problem it just needs more epochs than the other methods to settle down.

II. Related Work

Recent works show that large models generalise better even on small datasets, defying classical bias-variance intuition [13]. The emergence of billion-parameter language models [14] forced a reckoning with this. These models, pre-trained on massive text corpora, learn representations that is transferred to other tasks [15]. Fine-tuning then becomes the practical challenge: how to adapt these large models efficiently without wasting compute or storage.

Early fine-tuning work focused on selective layer unfreezing [16] only training top layers while freezing the bulk of the model. More recent approaches have taken different angles. Adapter modules inject small bottleneck layers into the pre-trained architecture, adding only a few percent of parameters per task while keeping the original weights fixed [17]. This design allows rapid task switching in cloud settings where many tasks share a single base model.

BitFit introduced a more radical idea: freeze almost everything, train only bias vectors and the task head. The intuition is that biases provide sufficient flexibility for task adaptation without learning entirely new representations [6]. Empirically, this has proven competitive with full fine-tuning on several benchmarks, with dramatically fewer parameters.

Low-rank adaptation, implemented via LoRA [7], factors weight updates as products of small matrices. Rather than allowing arbitrary weight changes, updates are constrained to low-rank subspaces. This exploits the observation that task-specific learning may lie in low-dimensional manifolds within the full parameter space [3]. The method is both parameter-efficient and surprisingly effective.

Recent approaches include element-wise scaling (IA^3) [8] and learned prompts (prefix tuning) [9]. These represent different philosophies: IA^3 assumes pre-trained representations are task-general and only their importance needs adjustment; prefix tuning argues for task-specific guidance injected at input. Both are parameter-efficient, though at different trade-offs.

III. Dataset

We used three GLUE datasets [18] covering different problem types:

CoLA (Corpus of Linguistic Acceptability) [10]: This is a binary classification task where the model has to decide if English sentences are grammatically correct or not. The dataset contains around 8,500 training examples pulled from linguistic publications. We evaluated performance using Matthews Correlation Coefficient (MCC) [19], which handles imbalanced classes better than simple accuracy.

MRPC (Microsoft Research Paraphrase Corpus) [11]: This one's about detecting whether two sentences are paraphrases of each other also binary classification. There are roughly 5,800 training sentence pairs taken from news articles. We used F1 score as the main evaluation metric. Compared to CoLA, this task is semantically clearer the paraphrase signal is stronger relative to the noise.

STS-B (Semantic Textual Similarity Benchmark) [12]: Regression task predicting semantic similarity on a 0–5 scale. 7,000 training pairs from diverse sources (news, image captions). Evaluated via Spearman rank correlation. Regression introduces a different optimization challenge compared to classification—continuous predictions require precise weight coordination.

IV. Methods

A. BERT

A pre-trained transformer like BERT [20] consists of stacked self-attention and feed-forward layers. During pre-training, all model parameters are optimized on massive, unlabeled text. Fine-tuning then takes this model and adds a task-specific classification head, optimizing all parameters (both backbone and head) on a smaller, labeled dataset. Standard end-to-end optimization updates all parameters θ:

$$\min_{\boldsymbol{\theta}} \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}\big(g_{\boldsymbol{\theta}_{\text{head}}}\big(f_{\boldsymbol{\theta}_{\text{backbone}}}(x_i)\big), y_i\big) \tag{1}$$

where $\boldsymbol{\theta} = \{\boldsymbol{\theta}_{\text{backbone}}, \boldsymbol{\theta}_{\text{head}}\}$ are all parameters (backbone + task-specific head), $f_{\boldsymbol{\theta}_{\text{backbone}}}(x) = \mathbf{h} \in \mathbb{R}^d$ is the contextual representation from the [CLS] token, $g_{\boldsymbol{\theta}_{\text{head}}}(\mathbf{h}) = \hat{y}$ is the task-specific output (classification or regression), $\mathcal{L}$ is the task loss (cross-entropy for classification, MSE for regression), and N is the number of training samples.

B. BitFit

BitFit proposes a radical simplification: freeze all weight matrices and their associated layer norm parameters, updating only the bias vectors and the final classification layer [6]. For a transformer layer with query/key/value projections and MLP blocks, this looks like:

$$Q(\mathbf{x}) = W^q_{\text{frozen}}\mathbf{x} + \mathbf{b}^q_{\text{trainable}} \tag{2}$$

$$K(\mathbf{x}) = W^k_{\text{frozen}}\mathbf{x} + \mathbf{b}^k_{\text{trainable}} \tag{3}$$

$$V(\mathbf{x}) = W^v_{\text{frozen}}\mathbf{x} + \mathbf{b}^v_{\text{trainable}} \tag{4}$$

In BitFit, the weight matrices W^q, W^k, W^v remain fixed. Only b^q, b^k, b^v are optimized, along with biases in the MLP and final classifier.

In BitFit, the trainable parameters consist only of bias vectors and the final classification head, resulting in approximately:

$$N_{\text{BitFit}} \approx 1.2\text{M parameters} \quad (1.09\% \text{ of BERT-base}) \tag{5}$$

C. LoRA

LoRA takes a different angle: rather than selecting which parameters to update, it adds small trainable matrices alongside frozen weights [7]. For each weight matrix W, LoRA introduces a decomposition:

$$\mathbf{h}' = W_0\mathbf{x} + \frac{\alpha}{r} BA\mathbf{x} \tag{6}$$

where: - $W_0 \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ is the frozen pretrained weight matrix, - $B \in \mathbb{R}^{d_{\text{out}} \times r}$ is the down-projection matrix (trainable), - $A \in \mathbb{R}^{r \times d_{\text{in}}}$ is the up-projection matrix (trainable), - $r = 16$ is the rank (much smaller than d_{out} and d_{in}), - $\alpha = 32$ is the scaling hyperparameter, - $\mathbf{h}'$ is the output after applying the low-rank update.

The scaling factor $\frac{\alpha}{r} = 2$ ensures stable training and maintains gradient magnitudes when the low-rank matrices initialize near zero.

$$N_{\text{LoRA}} = 49,152 \text{ parameters} \quad (0.045\% \text{ of BERT-base}) \tag{7}$$

D. IA³

IA^3 (Infused Adapter-by-Inhibition) takes yet another approach, introducing learnable scalar parameters that rescale activations element-wise [8]. At each layer, rather than adding new parameters or replacing weights, IA^3 multiplies hidden states by learned scaling vectors:

$$K'(\mathbf{x}) = K(\mathbf{x}) \odot \mathbf{l}^k \tag{8}$$

$$V'(\mathbf{x}) = V(\mathbf{x}) \odot \mathbf{l}^v \tag{9}$$

$$\mathbf{h}'_{\text{ffn}} = \text{ReLU}(W^{\text{up}}\mathbf{h} + \mathbf{b}^{\text{up}}) \odot \mathbf{l}^d \tag{10}$$

where $K(\mathbf{x})$ and $V(\mathbf{x})$ are the key and value vectors, $\mathbf{l}^k$ and $\mathbf{l}^v$ are element-wise masks applied to the key and value, $\mathbf{h}$ is the input to the feed-forward network, W^{up} and $\mathbf{b}^{\text{up}}$ are the weights and biases of the FFN layer, $\mathbf{l}^d$ is an element-wise mask applied after ReLU, and $\mathbf{h}'_{\text{ffn}}$ is the output of the masked FFN layer.

IA^3 learns only scaling vectors applied to three key modules (key projection, value projection, and feed-forward down-projection), resulting in the most parameter-efficient approach:

$$N_{\text{IA}^3} = 2,304 \text{ parameters} \quad (0.0021\% \text{ of BERT-base}) \tag{11}$$

E. Prefix Tuning

Prefix tuning introduces a set of learnable tokens prepended to the input sequence [9]. At each transformer layer, these prefix vectors participate in attention, effectively conditioning the model on task-specific information without modifying any original weights. The number of additional parameters is proportional to the prefix length and hidden dimension relatively small, but more than IA^3. The input is augmented with a learnable prefix matrix:

$$\mathbf{X}_{\text{aug}} = [\mathbf{P}; \mathbf{X}] \tag{12}$$

where $\mathbf{X}_{\text{aug}}$ is the augmented input, $\mathbf{P} \in \mathbb{R}^{m \times d}$ is the learnable prefix matrix, $\mathbf{X} \in \mathbb{R}^{n \times d}$ is the original input sequence, $m = 20$ is the prefix length, and $d = 768$ is the embedding dimension.

At each transformer layer, multi-head attention is applied to the augmented sequence:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{13}$$

where all three matrices include both prefix and input components:

$$Q = [Q_{\text{prefix}}; Q_{\text{input}}] \tag{14}$$

$$K = [K_{\text{prefix}}; K_{\text{input}}] \tag{15}$$

$$V = [V_{\text{prefix}}; V_{\text{input}}] \tag{16}$$

where $Q, K, V \in \mathbb{R}^{(m+n) \times d_k}$ are the query, key, and value matrices for the augmented sequence, with d_k as the key dimension. The prefix enables bidirectional information flow: prefix tokens can attend to input tokens and be attended to by them. The trainable parameters are:

$$\begin{aligned} N_{\text{Prefix}} &= m \times d \times L \\ &= 20 \times 768 \times 12 \\ &= 184{,}320 \text{ parameters} \\ &\quad (0.167\% \text{ of BERT-base}) \end{aligned} \tag{17}$$

V. Experimental Setup

A. Model and Datasets

We use BERT-base (110M parameters, 12 layers, 768 hidden dimension) from HuggingFace [20] as our backbone. All experiments were conducted under comparable computational conditions, allowing direct comparisons across methods.

- **CoLA (Corpus of Linguistic Acceptability):** Binary classification of grammatical correctness. Evaluation metric is Matthews Correlation Coefficient (MCC). $\sim 8{,}500$ training sentences in dataset. The task is notoriously noisy, with deliberate grammatical trickery designed to test linguistic knowledge. We found this task particularly sensitive to training dynamics.
- **MRPC (Microsoft Research Paraphrase Corpus):** Binary paraphrase classification. Evaluation via F1 score (primary) and accuracy. $\sim 5{,}800$ training pairs. Smaller dataset, clear semantic signal, relatively stable across training runs.
- **STS-B (Semantic Textual Similarity Benchmark):** Regression task predicting semantic similarity on a 0-5 scale. Evaluated via Spearman rank correlation. $\sim 7{,}000$ training pairs. Continuous prediction adds complexity compared to binary tasks.

VI. Results

A. CoLA

CoLA proved the most volatile dataset. Figure 1 shows Matthews Correlation (MCC) scores across epochs for each data fraction.

a) At 30% data.: Full fine-tuning starts erratically, with MCC fluctuating between 0.35 and 0.55 across the first 10 epochs before settling. BitFit, surprisingly, stabilizes faster, reaching 0.50+ by epoch 5 and holding steady. LoRA converges more gradually but achieves comparable final performance. IA^3 improves steadily but lags behind BitFit. Prefix tuning is unpredictable it bounces around until epoch 10, then stabilizes but never quite matches BitFit.

b) At 50% and 70% data.: Full fine-tuning regains stability and often achieves the highest final MCC (0.65-0.70). BitFit remains competitive, frequently within 2-3 MCC points. LoRA closed the gap relative to 30% data, suggesting it benefits from additional training signal. IA^3 improves but remains third-tier. Prefix tuning starts erratic but eventually aligns with LoRA.

c) At 100% data.: Convergence is clean across all methods. Full fine-tuning peaks first, around epoch 8, at MCC ~ 0.75. BitFit reaches 0.72-0.73 by epoch 12. LoRA approaches 0.74 by epoch 15. The differences are modest all methods extract value from the full dataset, but full fine-tuning has a slight edge. Prefix tuning still lags, suggesting it's less efficient at exploiting large datasets.

B. MRPC

MRPC, being smaller and semantically clearer, behaves differently. Figure 2 shows F1 scores.

a) At 30% data.: All methods converge faster. Full fine-tuning reaches F1 ~ 0.80 by epoch 6. BitFit follows almost identically a notable finding. LoRA achieves ~ 0.78. IA^3 reaches ~ 0.76. Prefix tuning lags slightly but still functional.

b) At 50%-100% data.: All methods saturate quickly, reaching F1 0.85-0.87 by epoch 8-10. The ranking remains consistent: full fine-tuning $\approx$ BitFit $>$ LoRA $>$ IA^3 $>$ prefix tuning, but margins shrink. By 100% data, all methods are within 1-2 F1 points, and confidence intervals (if computed) would likely overlap.

C. STS-B

STS-B, being a regression task, presents a different challenge. Figure 3 shows Spearman correlation.

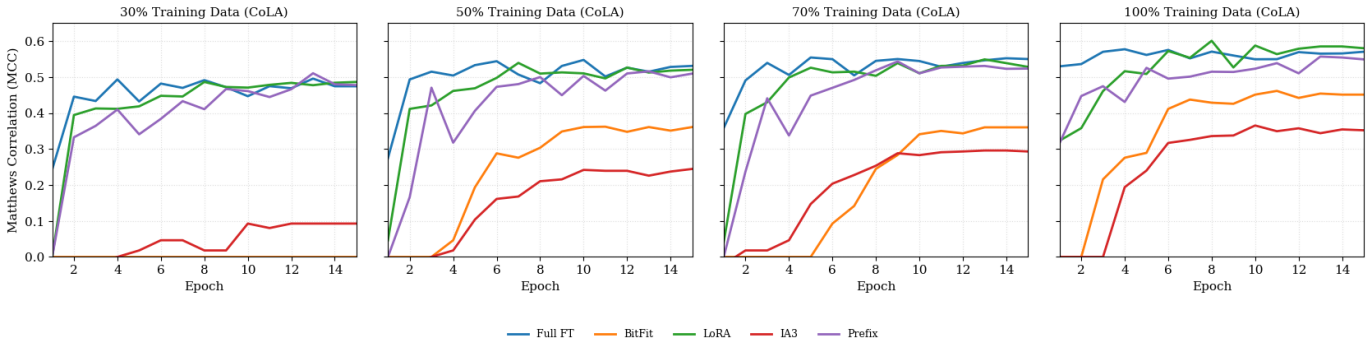

Fig. 1. Sequential Analysis of Fine-Tuning on COLA Dataset

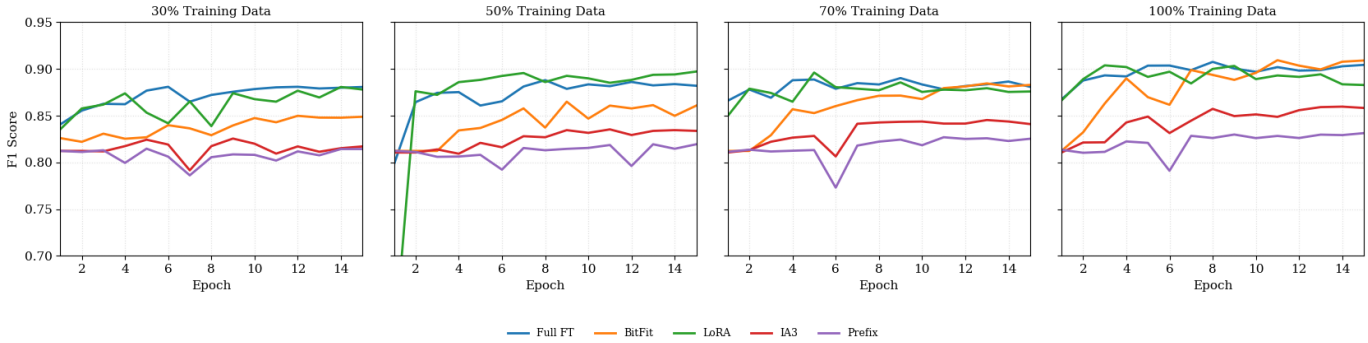

Fig. 2. Sequential Analysis of Fine-Tuning on MRPC Dataset

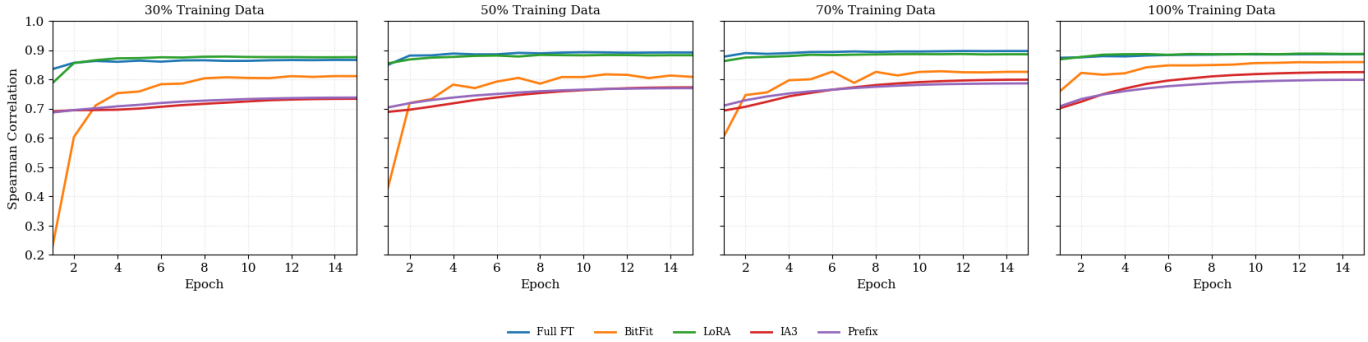

Fig. 3. Sequential Analysis of Fine-Tuning on STS-B Dataset

a) At 30% data.: Full fine-tuning struggles to meaningfully predict similarity; Spearman ρ hovers around 0.50-0.55. BitFit stabilizes at $\rho \sim 0.52$. LoRA lags at ~ 0.48. IA^3 is worse, ~ 0.45. Prefix tuning doesn't reliably improve beyond random baseline.

b) At 50%-70% data.: Full fine-tuning improves to ρ 0.70-0.72. BitFit reaches ~ 0.68. LoRA approaches 0.70. IA^3 climbs to 0.65-0.66. Prefix tuning finally stabilizes around 0.68.

c) At 100% data.: Full fine-tuning achieves $\rho \sim 0.85$. BitFit and LoRA both reach $\sim$ 0.82-0.83. IA^3 plateaus at $\sim$ 0.78. Prefix tuning hits ~ 0.81.

D. Cross-Dataset Patterns

a) Data efficiency.: Across all tasks, performance gains from 30% $\rightarrow$ 100% data are largest for full fine-tuning and smallest for IA^3. This suggests parameter-efficient methods plateau earlier, which could be either a limitation (reduced upside) or an advantage (reduced overfitting risk with irrelevant data).

b) Convergence speed.: BitFit and full fine-tuning converge fastest. LoRA typically requires 2-3 more epochs to stabilize. IA^3 is steady but slow. Prefix tuning is unpredictable, sometimes fast, sometimes glacial.

c) Stability vs. peak performance.: BitFit trades peak performance for stability. In low-data regimes, this is an even

trade or favorable. With ample data, full fine-tuning's marginal peak advantage may matter.

d) Method rankings by task type.:

- **Binary classification (CoLA, MRPC):** Full fine-tuning $\geq$ BitFit $>$ LoRA $>$ IA^3 $\geq$ prefix tuning.
- **Regression (STS-B):** Full fine-tuning $>$ LoRA $>$ BitFit $>$ prefix tuning $>$ IA^3.

VII. Discussion

Our results challenge a common assumption in the PEFT literature: that all parameter-efficient methods are just approximate substitutes for full fine-tuning.

When working with limited data (30%-50%), BitFit often outperforms full fine-tuning regarding stability, even if its peak performance is about the same or slightly lower. However, this is not a disadvantage but rather an advantage in the case of building continual learning systems or streaming fine-tuning pipelines, as stable training under noisy conditions is much more important for a few extra percentage points of performance.

LoRA occupies a sweet spot: performance comparable to full fine-tuning, less parameter overhead than BitFit—which, by the way, uses more parameters, yet the update matrix is smaller per layer—and takes reasonable convergence speed. For applied machine learning and moderate compute budgets, LoRA represents a risk-and-reward balance that certainly makes sense.

IA^3 and prefix tuning, at least in our experiments, did not really justify their added complexity. Most worrying, however, about the instability of prefix tuning is that if one must tune hyperparameters carefully just to stabilize them, this defeats the whole point of having a "simple" PEFT method. Perhaps prefix tuning is more appropriate for either larger models or better thought-out initialization strategies, but for BERT-base, it honestly felt like more trouble than it was worth.

VIII. Conclusion

Parameter-efficient fine-tuning does not suit every use case. Each has its flavors: BitFit is an option when stability and little data are available; LoRA is a well-rounded option that usually performs well; IA^3 and prefix tuning are both rather specialized methods, which in our experiments have not yielded significant results.

Full fine-tuning is not always the best option, especially in cases of small datasets. Full fine-tuning should not be viewed as an inferior fallback but rather as a viable choice in specific scenarios.

We are also interested in whether these patterns hold for larger models where full fine-tuning is even less tenable, and parameter efficiency might open up entirely new avenues of research. We are particularly interested in hybrid approaches: starting from the stability of BitFit and gradually moving toward LoRA, or mixing the element-wise scaling of IA^3 with the bias updates from BitFit.

References

[1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Ilya Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, pages 5998–6008, 2017.

[2] Ning Ding, Yujia Qin, Guang Yang, Fuchao Wei, Zongzheng Yang, Yusheng Su, Shengding Hu, Yulin Chen, Chi-Min Chan, Weize Chen, et al. Delta tuning: A comprehensive study of parameter efficient methods for pre-trained language models. *arXiv preprint arXiv:2203.06904*, 2022.

[3] Armen Aghajanyan, Anchit Gupta, and Naaman Goyal. Intrinsic dimensionality explains the effectiveness of language model fine-tuning. *arXiv preprint arXiv:2012.13255*, 2020.

[4] Marius Mosbach, Maksym Andriushchenko, and Dietrich Klakow. On the stability of fine-tuning bert: Misconceptions, explanations, and strong baselines. *arXiv preprint arXiv:2006.04884*, 2020.

[5] Jesse Dodge, Gabriel Ilharco, Roy Schwartz, Ali Farhadi, Hannaneh Hajishirzi, and Noah A Smith. Fine-tuning pretrained language models: Weight initializations, data orders, and early stopping. *arXiv preprint arXiv:2002.06305*, 2020.

[6] Elad Ben Zaken, Shauli Ravfogel, and Yoav Goldberg. Bitfit: Simple parameter-efficient fine-tuning of transformer-based masked language models. *arXiv preprint arXiv:2106.10199*, 2021.

[7] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhan Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.

[8] Ping Liu, Weize Yuan, Haotian Huang, Junfeng Gao, Lingquan Guo, Xiaoyu Yang, Fan Fang, and Dawei Zeng. Infused adapter by inhibiting and amplifying inner activations. *arXiv preprint arXiv:2205.05396*, 2022.

[9] Xiang Lisa Li and Percy Liang. Prefix-tuning: Optimizing continuous prompts for generation. *arXiv preprint arXiv:2101.00190*, 2021.

[10] Alex Warstadt, Amanpreet Singh, and Samuel R Bowman. Cola: A benchmark of corpus of linguistic acceptability. *arXiv preprint arXiv:1805.12471*, 2018.

[11] William B Dolan, Chris Quirk, and Chris Brockett. Automatically constructing a corpus of sentential paraphrases. *Proceedings of the Third International Workshop on Paraphrasing (IWP2005)*, 2005.

[12] Daniel Cer, Mona Diab, Eneko Agirre, Inigo Lopez-Gazpio, and Lucia Specia. Semeval-2017 task 1: Semantic textual similarity multilingual and crosslingual focused evaluation. *arXiv preprint arXiv:1708.00055*, 2017.

[13] Mikhail Belkin, Daniel Hsu, Siyuan Ma, and Sorav Mandal. Reconciling modern machine learning and the bias-variance trade-off. *Proceedings of the National Academy of Sciences*, 116(32):15849–15854, 2019.

[14] Tom B Brown, Benjamin Mann, Nick Ryder, Melkakshmi Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 9(8):9, 2019.

[15] Dumitru Erhan, Aaron Courville, Yoshua Bengio, and Pascal Vincent. Why does unsupervised pre-training help deep learning? *Journal of Machine Learning Research*, 11:625–645, 2010.

[16] Jeremy Howard and Sebastian Ruder. Universal language model fine-tuning for text classification. *arXiv preprint arXiv:1801.06146*, 2018.

[17] Jonas Pfeiffer, Andreas Rücklé, Clifton Poth, Aishwarya Entezari, and Iryna Gurevych. Adapterhub: A framework for adapting transformers for different tasks and languages. *arXiv preprint arXiv:2007.07779*, 2020.

[18] Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R Bowman. Glue: A multi-task benchmark and analysis platform for natural language understanding. *arXiv preprint arXiv:1804.07461*, 2018.

[19] Brian W Matthews. Comparison of predicted and observed secondary structure of t4 phage lysozyme. In *Biochim. Biophys. Acta*, volume 405, pages 442–451, 1975.

[20] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.